Recursion: A fn which call itself again & again until a specific condition meet

① $6! = n!$ ✓

$6! \Rightarrow 6 * 5 \times 4 \times 3 \times 2 \times 1$

n ka factorial ✓

$n * (n-1)!$

$n * (n-1) * (n-2) * (n-3) \times \cdots 1$

✓

```
int fact(n)
{   if (n ≤ 1)
        return 1;

return n * fact(n-1)

}
```

$6! = 6 * 5!$

$30! \Rightarrow 30 * 29 \times$

$(1, n) \Rightarrow sum$

```
int sum(num, n)
{   if (num == n)
        retur n;

    return num + sum(num+1, n)

}
```

$\rightarrow (1, n)$

$1 + (2, n)$

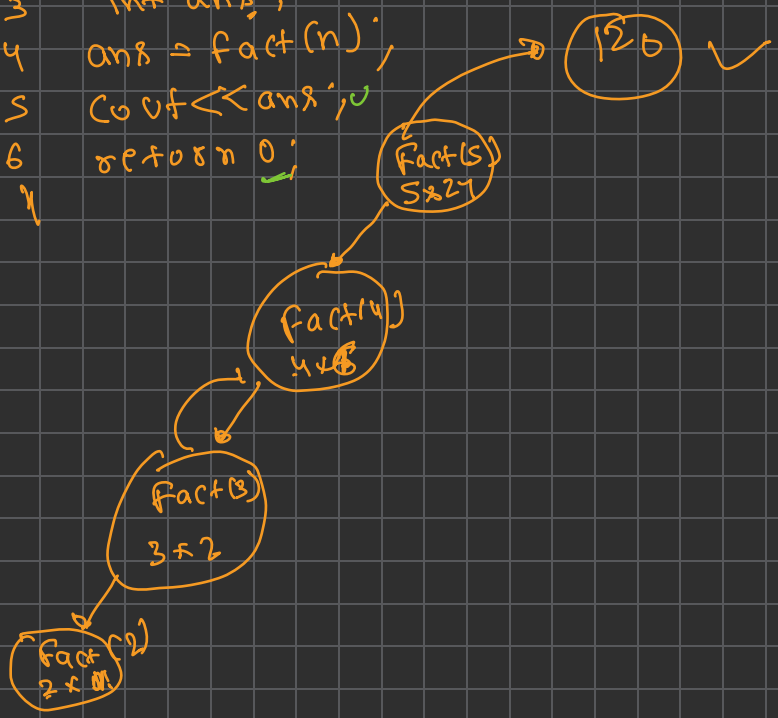
$1 + 2 + (3, n)$

```
void Sum(num, n)
{
    if (num==n+1)
        return;

    cout<< num + Sum(num+1, n);
}
```

$1 + \underline{Sum(2, n)}$

Behind;

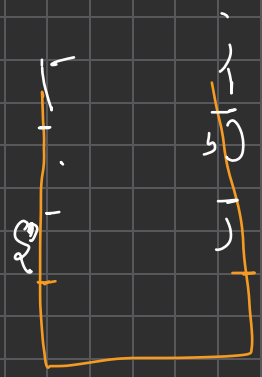
```
int fact(int n)
{
    if (n<=1)
        return 1;
    return n*fact(n-1)
}

int main()
{
    int n;
    cin>>n;
    int ans;
    ans = fact(n);
    cout<<ans;
    return 0;
}
```

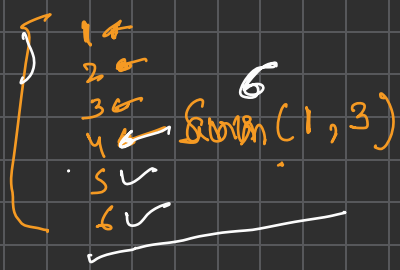


$\text{fart}(x)$

```
int sum( num, n )
{
    if (num==n)
        return n;

    int ans= num+sum(num+1,n)

    return ans;
}

int main()
{
    int n;
    cin>>n;
    int ans;
    ans= sum(1, n);
    cout<< ans
    return 0;
}
```

1 ✓
2 ✓
3 : 1+5
4 : 6





① 0 1 1 2 3 5 8 13 21 . - - - -

0th	1st	2nd	3rd	4th	5th	6th	7th	-

$5^{th} = 4^{th} + 3^{rd}$

$4 \Rightarrow 3^{rd} + 2^{nd}$

$3 \Rightarrow 2^{nd} + 1^{st}$

$2^{nd} = 1, 0$

$\left[\begin{array}{l} 1 \Rightarrow 1 \\ 0 \Rightarrow 0 \end{array}\right] \Rightarrow$ Base

$$\left[n^{th}\ \text{fib} = (n-1)\,\text{fib} + (n-2)\,\text{fib} \right]$$

$$\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$$

$$\text{fib}(2) = \text{fib}(1) + \text{fib}(0)$$

$$\left[\begin{array}{l} \text{fib}(1) = 1 \\ \text{fib}(0) = 0 \end{array}\right] \Rightarrow \text{Base condition}$$

```
int  fib(n)
{      if (n ≤ 1)
           return n;

       int ans = fib(n-1) + fib(n-2)
           return ans;
}
```

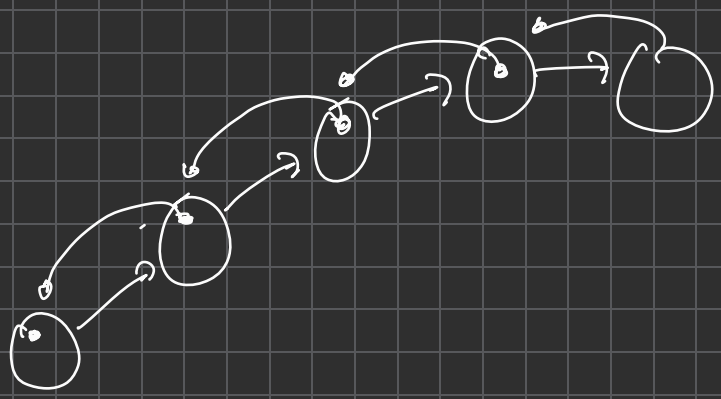
$fib(3) = 1 + 1 = 2$

$fib(2) = 1 + 0$

$fib(1) = 1$

$fib(0) = 0$

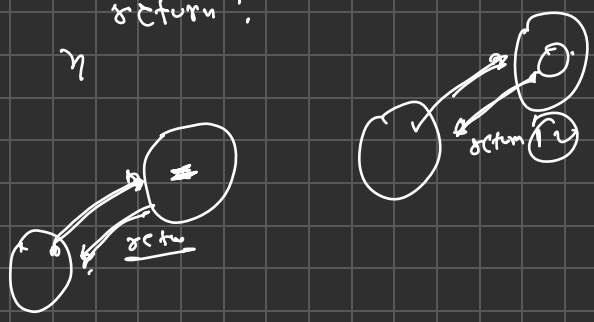
return;



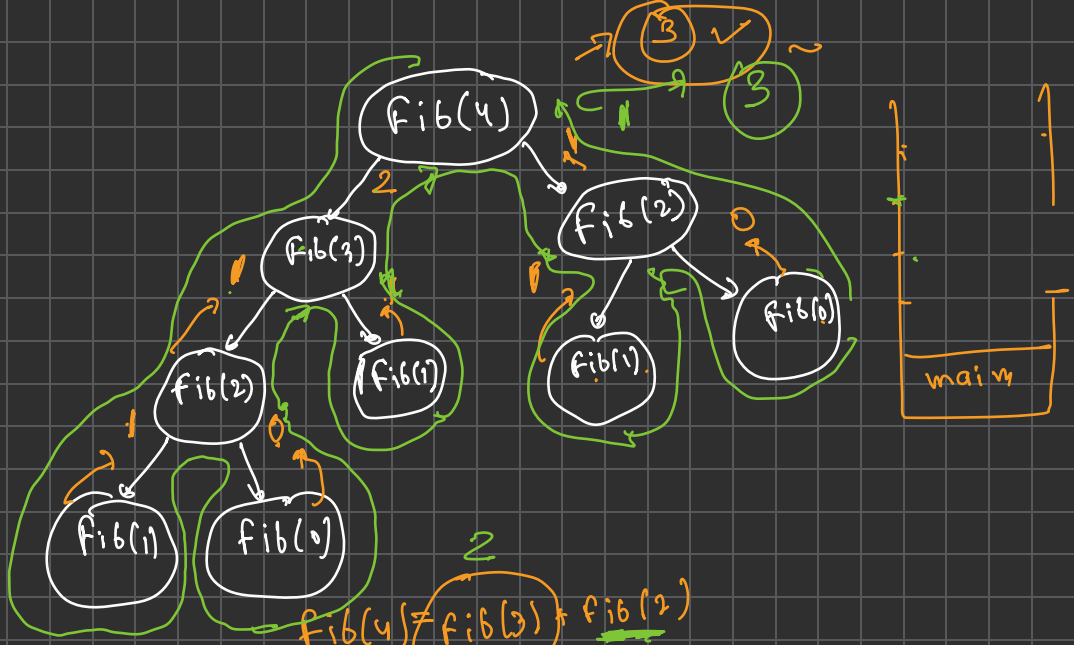
$\boxed{\text{Print}(1, n)}$

1
2
3
⋮
n

```
void Print (num, n)
{   if(num == n+1)
        return;
    cout << num;
    Print(num+1, n);
    return;
}
```



$fib(n) = fib(n-1) + fib(n-2)$



$$fib(4) = \underbrace{fib(3)}_{2} + \underbrace{fib(2)}_{1}$$

$$fib(3) = fib(2) + fib(1)$$

$$fib(2) = \underbrace{fib(1)}_{1} + \underbrace{fib(0)}_{0}$$

$$fib(1) = 1$$

$$fib(0) = 0$$

$n = 4$ → 1

1, 2 ✓

5 method

1 + 1 + 1 + 1 ⇒ ✓

1 + 1 + 2 → ✓

1 + 2 + 1 ⇒ ✓

2 + 1 + 1 = ✓

2 + 2 = ✓

$n = 1$

Total = $n = 1$, $n = 2$, $n = 3$, $n = 4$, $n = 5$

Top = 1 Top = 2, Top = 3, Top = 5, Top = 8

2 1

1 1 } 2 1 1 1

2 1 2

$n = 5$,

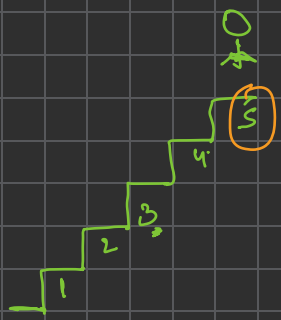
- $1+1+1+1+1$
- $1+1+1+2$
- $1+1+2+1$
- $1+2+1+1$
- $2+1+1+1$
- $2+2+1$
- $2+1+2$
- $1+2+2$

n	ans
1	1
2	2
3	3
4	5
5	8
6	13

$$\text{Climb}(n) = \text{Climb}(n-1) + \text{Climb}(n-2)$$

```
if (n <= 2)
    return n;
```

$$\text{Climb}(5) = \text{Climb}(4) + \text{Climb}(3)$$



Total(4):

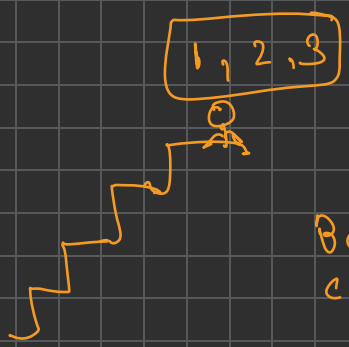
$$\boxed{5} = \begin{bmatrix} 1+1+1+1+1 \\ 1+1+2+1 \\ 1+2+1+1 \\ 2+1+1+1 \\ 2+2+1 \end{bmatrix}$$

Total(3)

$$\begin{bmatrix} 1+1+1+2 \\ 1+2+2 \\ 2+1+2 \end{bmatrix} = 5$$

$$\Rightarrow 3 \begin{bmatrix} \boxed{1+1+1+1+1} \\ 1+2+1+1 \\ 2+1+1+1 \end{bmatrix} \Rightarrow 3.$$

11



1, 2, 3

n	step
1	1
2	2
3	4
4	7
5	13
6	

Base case

```
climb(6)
    if (n == 3)
        return 4
    if (n <= 2)
        return n;
```

- 1 1 1 1
- 1 1 2
- 1 2 1
- 2 1 1
- 2 2
- 1 3
- 3 1

$$climb(n) = climb(n-1) + climb(n-2) + climb(n-3)$$